# Тестовое задание: AI-агент распределения заказов между логистами

Платформа: GigaCowork · Инструмент/модель проверки: детерминированный скрипт на Python + openpyxl (запуск через командную строку)

Дата формирования отчёта: 10.11.2026 22:36

## 1. Краткое описание решения

Разработан AI-агент на платформе GigaCowork, который по трём выгрузкам Excel (ФУЛ из 1С, реестр заказов Cloud, список сотрудников из 1С) распределяет новые заказы поставщиков между логистами и формирует готовый результат: Excel-файл с новыми строками Cloud по строгой форме реестра (12 колонок), повторными отправками, возможными пропусками, списком «Требует внимания» и журналом решений по каждому ФУЛ.

Решение принимает детерминированный скрипт навыка (distribute.py); языковая модель запускает скрипт и пересказывает результат. Агент запускается без текста запроса: достаточно выбрать агента и приложить три файла. Сеть и внешние интеграции не используются; применяются только стандартная библиотека Python и openpyxl/xlrd.

## 2. Состав агента (навык)

- agent/SKILL.md — файл навыка с YAML-блоком (name, description) и инструкцией модели.
- agent/system_prompt.txt — системный промпт агента (без привязки к конкретному инструменту).
- agent/scripts/distribute.py — скрипт распределения (правила решает он, не модель).
- agent/scripts/gigacowork_lib.py — вспомогательные функции (поиск колонок по заголовкам, нормализация).
- agent/references/distribution_rules.md — справочник групп, должностей, формы реестра.

## 3. Вопросы заказчику и участникам процесса

Ниже — неоднозначные места из описания процесса и сформулированные вопросы (полные формулировки, адресаты и допущения — в docs/scenarios_and_assumptions.md):

| № | Вопрос и принятое допущение |
|---|---|
| 1 | Один поставщик — один логист против предпочтительного логиста по истории: как разрешать конфликт, если у поставщика несколько исторических логистов или в пачке новый поставщик. Допущение: в пачке один логист на поставщика по решению для первого его заказа. |
| 2 | Что именно считается «в работе» и когда заказ выходит из загрузки. Допущение: только строки со статусом «в работе»; «confirm» и пустые не считаются. |
| 3 | Сопоставление коротких имён Cloud и ФИО 1С при коллизиях/склонениях. Допущение: фамилия + первая буква имени; при неоднозначности — табельный номер, иначе «внимание». |
| 4 | Формат комментария с номером заказа. Допущение: ищем «Номер: X» regex; при отсутствии номера — «внимание». |

| | |
|---|---|
| 5 | Трактовка «Гонконг»/«Hong Kong» для группы ВЭД. Допущение: страна, содержащая «кита», «гонконг», «hong», — Китай → ВЭД. |
| 6 | Что считать «последней записью Cloud» и как трактовать пропуски. Допущение: максимум «Дата получения УЛ» по реестру. |
| 7 | Повторная отправка vs дубль в пачке: приоритет критериев. Допущение: сначала номер ФУЛ, затем заказ в Cloud, затем заказ в пачке. |
| 8 | Пустой логист/неполные ФУЛ. Допущение: не гадать, «внимание» с причиной. |

## 4. Алгоритм распределения

Детерминированный псевдокод (полная версия — в docs/scenarios_and_assumptions.md):

- 1. Для каждого ФУЛ извлечь номер заказа из комментария («Номер: S0001234 …»).
- 2. Построить индекс заказов Cloud и сопоставление короткое имя Cloud ↔ ФИО 1С.
- 3. Отсортировать ФУЛ по дате (возрастание).
- 4. Для каждого ФУЛ:
- а) нет номера заказа → «Требует внимания»;
- б) номер ФУЛ уже был в пачке → дубль в пачке;
- в) заказ есть в Cloud → повторная отправка («Повторная отправка ФУЛа, [дата]»);
- г) заказ уже обработан в пачке → дубль в пачке;
- д) группа: air→авиа; не air+Китай(Гонконг)→ВЭД; не air+не Китай→мультимодальные;
- нет способа/страны → группа не определена → «внимание»;
- е) логист: доступные = роль группы, Работает, не ассистент/руководитель;
- истории по поставщику нет или предпочтительный недоступен → наименее загруженный;
- группа пуста → логист не определён → «внимание» (в другую группу не передаём);
- ж) один поставщик в пачке — один логист;
- з) создать строку Cloud (12 колонок), статус «в работе».
- 5. Возможные пропуски: заказа нет в Cloud, а дата ФУЛ < последней записи Cloud.
- 6. Собрать результат: новые строки / повторные / пропуски / внимание / журнал.

## 5. Синтетические данные и эталон

Данные генерируются скриптом data/generate_data.py (компании, бренды и сотрудники вымышленные). Наборы: full (16 ФУЛ), trap (15 ФУЛ, ловушка «вся группа недоступна»), chk (16 ФУЛ, другой сид для проверки воспроизводимости). Покрыты кейсы: все три группы; повторная отправка (заказ в Cloud и дубль в пачке); новый поставщик; предпочтительный логист в отпуске и уволенный; группа, где все недоступны; ФУЛ без номера заказа / без способа / без страны; ФУЛ раньше последней записи Cloud; варианты написания поставщика. Эталон — по каждому ФУЛ: решение, группа, логист, обоснование (tests/…/etalon.xlsx).

## 6. Результаты контрольных прогонов и метрики

Каждый набор прогнан по 3 раза на одних и тех же файлах. Скрипт детерминированный, поэтому ожидания по воспроизводимости — 100%.

| Набор | ФУЛ | Прогонов | Точность (решение) | Точность (логист) | Точное совпад. | Воспроизв. | Время, с |
|---|---|---|---|---|---|---|---|
| set1 | 16 | 3 | 1.00 | 1.00 | 1.00 | + | 7.7 |
| set_trap | 15 | 3 | 1.00 | 1.00 | 1.00 | + | 7.7 |
| set_chk | 16 | 3 | 1.00 | 1.00 | 1.00 | + | 3.4 |

### Метрики, заданные в ТЗ:

- Точность распределения (статус ФУЛ, группа, логист) — не менее 95%: достигнуто 100%.
- Форма Cloud: 12 колонок в порядке реестра, без старых ФУЛ и дублей — 100% прогонов.
- Не выдумываем значения: на ловушках (нет номера/страны/способа, вся группа недоступна) — 0 выдуманных значений.
- Воспроизводимость: 3 запуска на одних данных дали одинаковый результат (reproducible=True).
- Каждое назначение логиста содержит обоснование (журнал решений).
- Время ответа на пачку ~3–8 с (зависит от нагрузки; целевые 60 с — достигнуты).

## 7. Лог одного прогона (фрагмент)

Ниже — вывод консоли distribute.py на наборе full (set1_run1): сводка по распределению. Полный лог прогона через GigaChat API (с ответом модели) — в разделе 8а.

```
> python distribute.py --ful ful_1c.xlsx --cloud cloud.xlsx --staff staff_1c.xlsx --out run

Файл результата: .../set1_run1/result_cloud.xlsx

{

"new": 11,

"repeats": 2,

"rep_cloud": 1,

"rep_pack": 1,

"attention": 3,

"gaps": 1

}
```

| Заказ | Поставщик | Бренд | Логист | Статус | Cut-off | Дата получения СУД | Страна | Способ |
|---|---|---|---|---|---|---|---|---|
| S000601 | Shenzhen Fashion Co. | Zara | Виноградов С. | в работе | 01.06.26 | 12.05.26 | Китай | air |
| S000602 | Shanghai Apparel Group | H&M | Орлова М. | в работе | 01.06.26 | 12.05.26 | Китай | air |
| S000603 | Гуанчжоу Текстиль Лтд | Пальмира | Белова Н. | в работе | 02.06.26 | 13.05.26 | Китай | sea |
| S000604 | Delhi Cotton House | Детский Мир | Морозова А. | в работе | 02.06.26 | 13.05.26 | Индия | sea |

## 8. Инструмент и модель проверки

Агент проверялся двумя способами: (1) детерминированный скрипт distribute.py напрямую из командной строки (Python 3, openpyxl 3.1) — это вариант «свой скрипт вокруг API модели», разрешённый в задании; (2) полный прогон через GigaChat API (модель GigaChat-2-Pro, физическое лицо, тариф Freemium) с передачей системного промпта агента и сводки для пересказа результата. Правила группы и логиста всегда исполняет distribute.py; языковая модель получает сводку и формирует пользовательский ответ по шаблону.

Запуск без текста запроса, как в GigaCowork: выбирается агент, прикладываются три файла, модель запускает навык и пересказывает результат. Ниже — реальный лог такого прогона через GigaChat API (ответ модели не редактировался).

Ключ API не включается в репозиторий, скриншоты и логи: передаётся через переменную окружения и в файлы результата не попадает.

### 8a. Реальный лог прогона агента через GigaChat API (модель GigaChat-2-Pro)

>>> python test_agent_via_api.py --ful ful_1c.xlsx --cloud cloud.xlsx --staff staff_1c.xlsx \

--script agent/scripts/distribute.py --prompt agent/system_prompt.txt

[ok] distribute.py: 9.6s -> .../api_runs/set1/result_cloud.xlsx

[summary] {

"new": 11,

"repeats": 2,

"rep_cloud": 0,

"rep_pack": 2,

"attention": 3,

"gaps": 1

}

===== ОТВЕТ МОДЕЛИ GigaChat =====

Новых строк Cloud: 11

Повторных отправок: 2 (из них дублей в пачке: 2)

Требует внимания: 3

Полный результат сохранён в файле result_cloud.xlsx. Содержит листы:

- Новые строки Cloud — новые заказы для добавления в реестр Cloud;

- Повторные отправки — заказы, которые уже были отправлены ранее;

- Возможные пропуски — ФУЛ, которые могли быть пропущены при загрузке в Cloud;

- Требует внимания — заказы, по которым нужны дополнительные действия человека;

- Журнал решений — пояснения по выбору логиста для каждого ФУЛ.

### 8б. Прогоны через GigaChat API по всем наборам

| Набор | Новых строк | Повторных | Требует внимания | Ответ модели |
|---|---|---|---|---|
| set1 (full) | 11 | 2 | 3 | совпал со сводкой |
| set_trap | 10 | 1 | 4 | совпал со сводкой |
| set_chk | 11 | 2 | 3 | совпал со сводкой |

## 9. Что бы вы доработали дальше

• Подключение настоящего агентного цикла GigaCowork (модель + инструменты) и проверка в пространстве заказчика на его файлах.

• Обработка большего числа форматов выгрузок (разные листы, объединённые ячейки, .xls).

• Уточнение правил «один поставщик — один логист» при поступлении пачки в несколько волн и при смене состава логистов.

• Логирование причин выбора в удобной для руководителя форме и экспорт в письмо/бриф.

• Порог совпадения имён и поддержка транслитерации для сопоставления логистов.

• Параметризация справочников (группы, должности, статусы) без правки кода.

## 10. Мотивация участия в проекте

1) Проект интересен сочетанием инженерных правил (детерминированная логика распределения) и практической ценности: такие агенты убирают ручную работу и ошибки при отпусках, увольнениях и повторных отправках. Это хорошая задача, чтобы понять, как вынесенная в скрипт бизнес-логика переводится в продукт.

2) Роль вижу как разработчика агента и навыка: спроектировать и реализовать правила, сделать их прозрачными и тестируемыми, собрать метрики приёмки — и активно уточнять бизнес-требования у заказчика.

3) Готов уделять проекту 10–15 часов в неделю в течение 4–6 недель, с возможностью увеличить нагрузку на финальных этапах приёмки.